

EXP.NO: 7

Producer-Consumer Problem (Semaphores)

DATE: 06/03/2023

AIM:

To implement the Producer-Consumer problem using POSIX semaphores in C to synchronize access to a shared bounded buffer, ensuring mutual exclusion and preventing race conditions.

ALGORITHM:

1. Start the experiment.
2. Initialize three semaphores: mutex = 1, empty = BUFFER_SIZE, full = 0.
3. Producer thread: sem_wait(empty) → sem_wait(mutex) → produce item and add to buffer → sem_post(mutex) → sem_post(full).
4. Consumer thread: sem_wait(full) → sem_wait(mutex) → remove item from buffer → sem_post(mutex) → sem_post(empty).
5. Use pthread_create() to run producer and consumer as concurrent threads.
6. Use pthread_join() to wait for both threads to finish.
7. Destroy semaphores using sem_destroy() after completion.
8. Compile with gcc using the -lpthread flag and execute.
9. Stop the experiment.

CODE:

```
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>
#include <semaphore.h>
#include <unistd.h>

#define BUFFER_SIZE 5
#define ITEMS      8

int buffer[BUFFER_SIZE];
int in = 0, out = 0;

sem_t mutex, empty_sem, full_sem;

void *producer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&empty_sem);
        sem_wait(&mutex);

        buffer[in] = i;
        printf("Produced: %d at index %d\n", i, in);
        in = (in + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&full_sem);
    }
}
```

```

        sleep(1);
    }
    return NULL;
}

void *consumer(void *arg) {
    for (int i = 1; i <= ITEMS; i++) {
        sem_wait(&full_sem);
        sem_wait(&mutex);

        int item = buffer[out];
        printf("Consumed: %d from index %d\n", item, out);
        out = (out + 1) % BUFFER_SIZE;

        sem_post(&mutex);
        sem_post(&empty_sem);
        sleep(2);
    }
    return NULL;
}

int main() {
    pthread_t prod, cons;

    sem_init(&mutex, 0, 1);
    sem_init(&empty_sem, 0, BUFFER_SIZE);
    sem_init(&full_sem, 0, 0);

    pthread_create(&prod, NULL, producer, NULL);
    pthread_create(&cons, NULL, consumer, NULL);

    pthread_join(prod, NULL);
    pthread_join(cons, NULL);

    sem_destroy(&mutex);
    sem_destroy(&empty_sem);
    sem_destroy(&full_sem);

    printf("Producer-Consumer completed.\n");
    return 0;
}

```

OUTPUT:

```

[hp@localhost ~]$ gcc producer_consumer.c -o pc -lpthread
[hp@localhost ~]$ ./pc
Produced: 1 at index 0
Consumed: 1 from index 0
Produced: 2 at index 1
Produced: 3 at index 2
Consumed: 2 from index 1
Produced: 4 at index 3
Produced: 5 at index 4
Consumed: 3 from index 2
Produced: 6 at index 0

```

```
Produced: 7 at index 1
Consumed: 4 from index 3
Produced: 8 at index 2
Consumed: 5 from index 4
Consumed: 6 from index 0
Consumed: 7 from index 1
Consumed: 8 from index 2
Producer-Consumer completed.
```

RESULT:

Thus, the Producer-Consumer problem was successfully implemented using POSIX semaphores and pthreads. Synchronization was correctly achieved, preventing buffer overflow and underflow conditions.